Exact PTO Equivalence Universal Teleporting Agent

Annealed Controller

`analyses/annealing/DFST_Softmax/universal/hidden_30_data_150/`

1 Statement and Scope

Controllers. The exported table `minimized_transition_table.pdf` defines controller C, with initial state 0 and halt state 27. The function `universal_tp`$(\{\lambda, 0, 1\})$ defines expert $\mathcal{E}$, with initial state 0 and halt state 14. The adjacent expert export matches this function entry for entry. All tables retain the exported state numbers.

Input Domain. A simulated Turing machine has states $Q = \{0, \ldots, N-1\}$, $N \geq 1$, tape alphabet $\Gamma = \{\lambda, 0, 1\}$, and a total transition function

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}.$$

Motion S stops after writing. Every successor belongs to Q. The simulated machine starts in state 0 on input $v \in \{0, 1\}^*$. Use the exact `encode_tm` layout and initial head position described in Section 2.1. The domain permits arbitrary program sizes and finite, possibly empty binary inputs.

Execution Convention. The symbol λ denotes a blank. A transition (q', w, M) writes w, performs motion M, and enters state q'. Motions include the rune teleportations X, Y, Z. Section 2.1 specifies the grid coordinates, rune records, and hidden-symbol restoration.

Policy-Trajectory Observations. For program δ and input v, define the policy-trajectory observation (PTO) by

$$\mathrm{PTO}(\delta, v) = \big((r_t, w_t, M_t)\big)_t,$$

where r_t, w_t, and M_t denote the read symbol, written symbol, and motion at controller step t. A finite execution contributes the final S action exactly once, without halt-loop padding. Equality of infinite PTOs means equality of every finite prefix.

Theorem 1 (Exact PTO Equivalence)**.** *For every input in the stated domain,* $\mathrm{PTO}_C(\delta, v) = \mathrm{PTO}_{\mathcal{E}}(\delta, v)$. *The head positions, visible grids, and rune records agree at every step, including symbols hidden beneath runes. Either both controllers halt on the same step with the same final configuration, or both execute infinitely many steps with equal finite prefixes.*

The theorem concerns autonomous executions of the exported discrete controller, including empty input. The invariant below also proves faithful simulation of the encoded Turing machine.

2 Expert Behavior

2.1 Encoding and Rune Semantics

Let $\kappa = \max(1, \lceil \log_3 N \rceil)$. Encode each state with a fixed-width, most-significant-first ternary word using L, R, S for $0, 1, 2$ respectively. Write this word as $c(q)$. The trit symbol S encodes the value 2; the motion S stops execution.

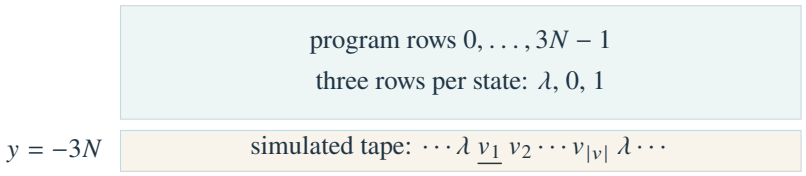
Order tape symbols as $\sigma_0 = \lambda < \sigma_1 = 0 < \sigma_2 = 1$. For $\delta(q, \sigma_i) = (q', w, M)$, program row $r = 3q + i$ contains

$$\big[\sigma_i, \lambda, c(q), 0, c(q'), 0, w, M\big]. \tag{1}$$

The source and successor codes each occupy κ separate cells. Thus the program has $3N$ rows, ordered by state and then by read symbol.

Columns	Field	Purpose
0	σ_i	Simulated tape read symbol
1	λ	Bookmark column for rune X
$2, \ldots, \kappa + 1$	$c(q)$	Source-state code
$\kappa + 2$	0	First delimiter
$\kappa + 3, \ldots, 2\kappa + 2$	$c(q')$	Successor-state code
$2\kappa + 3$	0	Second delimiter
$2\kappa + 4$	w	Symbol to write
$2\kappa + 5$	M	Simulated move L, R or S

Initial Placement. Row r occupies grid height $y = -r$, so D advances down the table and U moves up the table. Initially X covers the blank at $(1, 0)$; no cell contains Y or Z. The input occupies columns $1, \ldots, |v|$ of row $3N$, at height $-3N$. The controller starts at $(1, -3N)$, including for empty input. The tape row extends infinitely in both horizontal directions.



Rune Semantics. Each of X, Y, Z has one position and a hidden symbol. Writing a rune first restores the hidden symbol at the old rune position, then installs the rune over the symbol at the new position. Rewriting a rune in place preserves the hidden symbol. Writing an ordinary symbol at a rune removes that rune. A motion X, Y or Z jumps to the corresponding rune after the write. Ordinary motions move one cell; S stops without moving.

The *logical program* means the program with every rune replaced by the corresponding hidden symbol. Correctness depends on preserving this logical program, not on treating a rune as a destructive overwrite.

2.2 Endpoint and Prefix Invariant

Lemma 1 (Endpoint and Prefix Invariant)**.** *At the start of every simulated step, the expert occupies state* 0 *at the simulated tape head. Rune X occupies column* 1 *on the first or last row of the current simulated state's three-row block. After matching j successor trits, X occupies column* $1 + j$ *on a row with the correct length-j successor prefix. The row index remains congruent to* 0 *or* 2 *modulo* 3*. The logical program remains unchanged.*

Proof. Initially X occupies the first row of state 0. Assume the invariant at the start of a simulated step. The expert reads the tape symbol, installs Z, jumps to X, and steps left. The symbol search selects the matching row within the current state's three-row block. The following right scan crosses column 1 and the source code to the first delimiter, leaving X in place.

For successor trit t, the expert places Y on the trit, jumps to X, and steps right to the next source-code column. All rows sharing the matched prefix form one interval. Within that interval, next-trit values increase monotonically down the table. The successor state exists, so a nonempty subinterval contains next trit t. Moving down on a smaller trit, or up on a larger trit, reaches t before leaving the matched-prefix interval.

An immediate match preserves the endpoint. A downward search enters the target subinterval at the first row, with read symbol λ; an upward search enters at the last row, with read symbol 1. Both rows occupy endpoints of three-row state blocks. Writing X advances the matched prefix by one trit. Jumping to Y resumes instruction decoding.

After all κ trits, X occupies a row in the successor state's block. The second delimiter triggers a leftward rewind of X to column 1 on the same row. The expert then applies the prescribed write and motion at Z. Rune relocation restores every covered program symbol, preserving the logical program. Each nonstopping motion establishes the next simulated step. □

Lemma 2 (No Missing Middle Trit)**.** *Within a fixed source-code prefix block, a vertical search cannot pass directly between an L region and an S region. A nonempty R region separates the two regions.*

Proof. The state numbers fill the contiguous interval $0, \ldots, N - 1$, with a common encoding width. A prefix block contains a possibly truncated interval of consecutive integers. A block containing next-trit values L and S also contains every state in the intervening R subinterval. Each state contributes all three read-symbol rows. □

Corollary 1 (Faithful Expert Simulation)**.** *Each simulated TM transition takes finitely many controller steps and performs the write and motion prescribed by δ. The expert halts exactly when the simulated TM issues S.*

Proof. Lemma 1 selects the correct instruction and successor. Every scan stays within a finite program interval, and every trit search reaches a nonempty target interval. The final two instruction fields supply the prescribed write and motion at Z. Only a simulated S triggers a stopping transition. □

3 Observation Protocol

The protocol records finite geometric context. Let $e \in \{0, 2\}$ identify the endpoint row occupied by X. Use $r, j \in \{0, 1, 2\}$ to index tape symbols and $t, a \in \{L, R, S\}$ to denote trits. A vertical search records the last observed trit. Each phase and parameter tuple determines an expert state.

Phase	Expert state	Allowed reading and continuation
Dispatch(e)	0	Read σ_r; jump to selection of r.
Selection jump(e, r)	$1 + r$	Read X; step left to row index $j = e$.
Select(e, r, j)	$1 + r$	Read σ_j; if $j = r$, cross column 1; otherwise change j by $\operatorname{sgn}(r - j)$.
Cross(e, r)	4	Read X if $e = r$, and λ otherwise; enter first source trit.
First source trit(e)	4	Read any trit; enter remaining source-code scan.
Source scan(e)	4	Trit: repeat. Delimiter 0: enter first successor trit.
First successor trit(e)	5	Read a trit t; jump to X for matching t.
Successor scan(e)	5	Trit t: match t. Delimiter 0: start rewind.
Match jump(e, t)	g_t	Read X; step right and begin comparison.
First comparison(e, t)	g_t	Read a. If $a = t$, resume with endpoint e; if $a < t$, search down; if $a > t$, search up.
Search down(t, a)	g_t	Read a or the immediate successor of a. Repeat until t; then resume with endpoint 0.
Search up(t, a)	g_t	Read a or the immediate predecessor of a. Repeat until t; then resume with endpoint 2.
Resume(e, t)	g_t	Read Y; advance to successor scan with endpoint e.
Rewind jump(e)	9	Read X; step left.
Rewind(e)	9	Trit: continue left. Blank: move X here and jump to Y.
Rewind return(e)	9	Read Y; step right to the write field.
Read write(e)	10	Read σ_r; remember this write symbol.
Read motion(e, r)	$11 + r$	Read $M \in \{L, R, S\}$; jump to Z.
Apply(e, r, M)	$h_{r,M}$	Read Z; write σ_r and move M. Dispatch(e) if $M \neq S$; otherwise halt.
Halt	14	No outgoing action.

Here $g_L = 6$, $g_R = 7$, $g_S = 8$, and

$$h_{r,L} = 1 + 2r, \qquad h_{r,R} = 2 + 2r, \qquad h_{r,S} = 11 + r.$$

The protocol starts at Dispatch(0). Searches update the remembered trit after each change. Thus a downward search for S that has seen L must see R before S. The protocol permits unbounded repetition while retaining finite annotations.

Lemma 3 (Protocol Coverage)**.** *Every expert execution on a correct encoding follows this protocol.*

Proof. Endpoint selection and the following column-1 reading follow from Lemma 1. Both state codes have positive width. Prefix-block monotonicity and Lemma 2 give the allowed vertical-search readings and endpoint updates. Rewind reaches the blank in column 1, then Y, the write field, the motion field and Z. These exhaust the phases of one simulated step; induction gives coverage for every finite prefix of the execution. □

The protocol permits additional observation sequences, including unequal lengths for the two code scans. The certificate checks every permitted transition, so this overapproximation preserves the equivalence argument.

4 Simulation Certificate

4.1 Dispatch, Row Selection and Code Scanning

For phase i, write Q_i for the controller-state set specified in the following certificate subsections. The simulation relation pairs expert state g_i with controller state $q \in Q_i$, with identical grids, heads, and rune records.

Dispatch. For either endpoint e, phase Dispatch(e) has $Q_i = \{0\}$. On reading $\lambda, 0, 1$, state 0 writes Z, jumps to X, and enters controller state $23, 0, 22$, respectively. These actions match expert state 0.

All Six Row-Selection Cases. The following lists the readings from arrival at X through crossing column 1. The controller-state list gives the state *before each reading*. Each route ends in controller state 10 at the first source-code trit. The table represents endpoint e by the corresponding read symbol, λ or 1.

Endpoint	Wanted	Readings in order	Controller states in order
λ	λ	X, λ, X	$23, 10, 19$
λ	0	$X, \lambda, 0, \lambda$	$0, 20, 7, 25$
λ	1	$X, \lambda, 0, 1, \lambda$	$22, 7, 19, 25, 25$
1	λ	$X, 1, 0, \lambda, \lambda$	$23, 10, 10, 9, 22$
1	0	$X, 1, 0, \lambda$	$0, 20, 4, 25$
1	1	$X, 1, X$	$22, 7, 19$

Each route uses expert state $1 + r$ during selection and state 4 for the final column-1 reading. The X step moves left; comparisons move up or down toward the wanted symbol; the match and crossing steps move right. Every step preserves the read symbol. Substitution in the actual table verifies each route, including successor state 10.

Scan the Source-State Code. The first trit uses controller state 10; subsequent trits use states from $\{4, 18\}$. All of the following writes preserve the read symbol and move right:

Controller state	Read L	Read R	Read S	Read delimiter 0
10	18	4	4	excluded at first trit
4	18	4	4	25
18	18	4	18	25

Entries list successor states. Expert state 4 remains 4 on trits and enters 5 on the delimiter. Thus $Q_i = \{10\}$ at the first source trit, $Q_i = \{4, 18\}$ during the remaining source scan, and $Q_i = \{25\}$ during the successor scan, including the first successor trit.

Successor-Code Dispatch. Controller state 25 has the four relevant transitions

read	L	R	S	0
successor	7	26	15	9

and in every case writes Y and jumps to X. The corresponding expert transitions enter states $6, 7, 8, 9$. The delimiter case occurs only after at least one successor trit. The match jumps therefore use states $7, 26, 15$, and the rewind jump uses state 9, independently of the endpoint.

4.2 Trit Matching, Rewind and Tape Updates

Match One Successor Trit. The match-jump states 7, 26, 15 preserve X and move right, entering 19, 17, 16 respectively. The following table gives all comparison cases. Each entry lists (successor, written symbol, motion).

Context	State	Read L	Read R	Read S
Target L	19	$(18, X, \mathsf{Y})$	$(19, R, \mathsf{U})$	$(19, S, \mathsf{U})$
Target R	17	$(17, L, \mathsf{D})$	$(2, X, \mathsf{Y})$	$(17, S, \mathsf{U})$
Target S: first comparison	16	$(3, L, \mathsf{D})$	$(5, R, \mathsf{D})$	$(3, X, \mathsf{Y})$
Target S: previous trit L	3	$(3, L, \mathsf{D})$	$(5, R, \mathsf{D})$	excluded
Target S: previous trit R	5	excluded	$(5, R, \mathsf{D})$	$(3, X, \mathsf{Y})$

The corresponding expert states have indices 6, 7, 8. The exclusions follow from the protocol, especially Lemma 2. After equality, states 18, 2, 3 respectively read and preserve Y, then move right to state 25. These states represent Resume(e, L), Resume(e, R), and Resume(e, S). An immediate match retains the endpoint; a downward search selects endpoint 0, and an upward search selects endpoint 2.

Rewind X and Reach the Write Field. Expert state 9 corresponds to the following exact sequence:

Phase	Controller state	Reading	Transition
Rewind jump	9	X	$(14, X, \mathsf{L})$
Rewind through code	14	$t \in \{L, R, S\}$	$(14, t, \mathsf{L})$
Rewind reaches column 1	14	λ	$(19, X, \mathsf{Y})$
Rewind return	19	Y	$(8, Y, \mathsf{R})$

The last step enters expert state 10, so $Q_i = \{8\}$ in phase ReadWrite(e). On reading $\lambda, 0, 1$, state 8 preserves the symbol, moves right and enters 21, 9, 8 respectively; the expert transitions enter states 11, 12, 13.

Motion Decoding and Tape Update. For each row below, reading the selected motion writes Y, teleports to Z, and enters the listed controller state. That state then reads Z, writes the indicated symbol and performs exactly the selected move.

Write symbol	Motion-reading state	Apply L	Apply R	Apply S
λ	21	7	4	14
0	9	3	26	18
1	8	11	1	15

The L and R cases return to controller state 0, matching expert state 0. The S cases enter controller halt state 27, matching expert halt state 14. These establish all Apply phases and the terminal pair.

Together, the certificate subsections specify Q_i for every reachable protocol phase and verify every permitted transition. Only source-code scanning requires multivalued sets: $\{4, 18\}$ for each endpoint. All other phase–parameter combinations have a unique controller state.

The certificate contains 91 reachable parameterized phases and 93 related phase–controller pairs, including the terminal pair. For every allowed protocol edge $i \xrightarrow{x} j$ and every controller state $q \in Q_i$, the preceding certificate gives

$$\delta_{\mathcal{E}}(g_i, x) = (g_j, w, M), \qquad \delta_C(q, x) = (q', w, M), \qquad q' \in Q_j. \tag{2}$$

The 162 local checks cover 81 distinct nonterminal entries of the controller table. A transition reaches a halt state exactly on an edge to the terminal phase.

5 Equivalence Proof

Proof of Theorem 1. Initially both configurations have the same grid, head, and rune records, with control states 0 related by Dispatch(0). Suppose the relation holds before a controller step. Lemma 3 supplies an allowed read symbol x. Equal grids and heads give the same read symbol to both controllers. Equation (2) gives equal writes and motions and preserves the relation between successor states. The next grids and heads therefore agree. Rune installation, restoration, and teleportation act deterministically, so the rune records also agree.

Induction establishes equality of every finite execution prefix. The terminal condition permits a halt exactly on an edge to the terminal phase. Consequently, both controllers halt on the same step or both execute infinitely many steps with equal finite prefixes. Corollary 1 identifies the expert execution with the encoded TM computation. □

6 Verification and Artifacts

Execution Checks. The accompanying checker recomputes the full finite certificate and checks the CSV against the JSON controller export and the expert against `universal_tp`. Independent executions additionally cover all 729 one-state transition tables on all 15 words of lengths 0–3 (10,935 pairs), 128 random cases and 42 directed state-boundary cases. The checks compare 1,129,521 controller transitions and 46,633 simulated TM steps, including programs with 2,188 states and eight-trit encodings. The checks compare grids, heads, hidden rune data and a direct TM simulation. The suite contains 6,753 halting cases and 4,352 bounded-prefix cases. Bounded prefixes establish no eventual-halting or nonhalting conclusion. The universal claim rests on the invariant and induction, not those finite executions.

Verification. Run `python3 verify_equivalence.py --quick` from this proof directory. Omit `--quick` to rerun the execution checks. The adjacent files include the LaTeX source, parameterized protocol, checker, reference-source snapshots, and JSON verification report.

Artifact Identity. The title page gives the repository-relative case directory. This proof directory contains the controller and expert exports, reference code, protocol, checker, and saved verification report. The verification script stores or reads SHA-256 checksums for these artifacts. A checksum identifies the exact bytes of a file; the checker compares the stored and computed values to detect changes. These checks bind the certificate to the supplied exports. The invariants, protocol coverage, and local transition identity establish equivalence.